SCIENTIFIC COMPUTING
HOMEWORK

AYŞE ATİK
23050951014

CONTENT

1. Introduction

The purpose of this homework is to generate probabilistic text using Maximum Likelihood Estimation (MLE). We will build simple language models from a text corpus, estimate probabilities from observed data, and generate new text by sampling from these learned probability distributions.

By analyzing the mechanics of word transitions within a literary corpus, this assignment demonstrates how computational systems capture linguistic patterns. We explore the fundamental trade-offs between model complexity, computational efficiency, and the overall fluency and coherence of the generated artificial text.

2. Dataset and Preprocessing

The dataset used for this assignment consists of the complete text corpus of the book *"Harry Potter and the Sorcerer's Stone"*. Before training the language models, the raw text was converted into a structured format to eliminate noise and normalize the vocabulary.

The following sequential preprocessing pipeline was implemented:

1. **Lowercasing:** All characters were converted to lowercase to ensure that the model treats words like *"Harry"* and *"harry"* as identical tokens, reducing vocabulary redundancy.
2. **Punctuation Removal & Segmentation:** Unnecessary punctuation marks (such as commas, exclamation marks, and quotation marks) were stripped using regular expressions, while sentence-ending periods were utilized to identify structural boundaries.
3. **Tokenization:** The clean text was split into individual string elements (words) using whitespace delimiters.
4. **Sentence Boundary Markers:** Special structural tokens, namely <s> (start of sentence) and </s> (end of sentence), were explicitly inserted around each sentence. This allows the model to learn how sentences typically begin and end.

Text Preprocessing Example:

```python
# Sample text
raw_text = open("Harry_Potter.txt", "r", encoding="utf-8").read()

def preprocess_text(text):
    # 1. Convert the text to lowercase.
    text = text.lower()

    # 2. Remove unnecessary punctuation if needed.
    text = re.sub(r"[^\w\s\.]", "", text)

    # 3. Tokenize the text into words.
    sentences = text.split(".")
    processed_tokens = []

    for sentence in sentences:
        words = sentence.strip().split()
        if words:  # Skip empty sentences
            # 4. Add special tokens <s> and </s> to mark sentence boundaries.
            processed_tokens.append("<s>")
            processed_tokens.extend(words)
            processed_tokens.append("</s>")

    return processed_tokens

tokens = preprocess_text(raw_text)

print("--- PART 1: TEXT PREPROCESSING ---")
print(f"Raw text first 100 characters:\n{raw_text[:100].strip()}...\n")
print(f"Tokenized text first 15 tokens:\n{tokens[:15]}\n")
print("-" * 50)
```

Screenshot 1: Preprocessing code

```
PS C:\Users\ASUS\Desktop\25-26\Second Season\SENG306\SENG_306_Homework>  c:; cd 'c:\Users\ASUS\Desktop\25-26\Second Season\SENG3
\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '56908' '--' 'c:\Users\ASUS\Desktop\25-26\Second Season\SEN
--- PART 1: TEXT PREPROCESSING ---
Raw text first 100 characters:
Harry Potter and the Sorcerer's Stone

CHAPTER ONE

THE BOY WHO LIVED

Mr. and Mrs. Dursley, of n...

Tokenized text first 15 tokens:
['<s>', 'harry', 'potter', 'and', 'the', 'sorcerers', 'stone', 'chapter', 'one', 'the', 'boy', 'who', 'lived', 'mr', '</s>']
```

Screenshot 2: Preprocessing result

3. Unigram Model

The Unigram model operates under the strong independence assumption, meaning it assumes that the probability of a word occurring is entirely independent of any preceding or following words. It treats the corpus as a chaotic "bag of words" where each token is drawn in isolation.

Mathematically, the Maximum Likelihood Estimation (MLE) for a unigram probability is defined as:

$$P(w) = \frac{\text{count}(w)}{N},$$

Where count(w) represents the total frequency of the target word in the corpus, and N represents the total number of all tokens (including sentence markers).

During the training phase, the frequencies of all unique words were counted and divided by N. High-frequency structural markers and core characters yielded significantly higher MLE probabilities.

```python
# --- 2.1 Unigram Model ---
unigram_counts = collections.Counter(tokens) # generates a dictionary that shows us how many of each word there are
total_unigrams = sum(unigram_counts.values()) # calculates the total number of words in the text (including duplicates)

# MLE Probablity Calculation: P(w) = count(w) / N
unigram_probs = {
    word: count / total_unigrams for word, count in unigram_counts.items()
}
```

Screenshot 3: Unigram Model MLE probability calculation code

```
--- PART 2: EXAMPLE MLE PROBABILITIES ---
Unigram Examples:
  P('<s>') = 0.0623
  P('harry') = 0.0137
  P('potter') = 0.0011
```

Screenshot 4: Calculated Unigram Probability Samples

4. Bigram Model

The Bigram model relaxes the strict independence assumption of the unigram approach by introducing a local history window. It relies on the First-Order Markov Assumption, which posits that the probability of a word depends solely on the single word that immediately precedes it.

The MLE conditional probability for a bigram is calculated by dividing the joint frequency of a word pair by the total frequency of the history word:

$$P(w_i \mid w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}.$$

In Python, this was implemented using a nested dictionary structure acting as a conditional frequency matrix. For each unique word, the model keeps a distinct probability distribution over all possible next words.

```python
# --- 2.2 Bigram Model ---
bigram_counts = collections.defaultdict(collections.Counter) # Creates a nested (2-dimensional) dictionary structure.

for i in range(len(tokens) - 1):
    w1, w2 = tokens[i], tokens[i + 1] # Which word comes most often after which word?
    bigram_counts[w1][w2] += 1

# MLE Probablity Calculation: P(w_i | w_{i-1}) = count(w_{i-1}, w_i) / count(w_{i-1})
bigram_probs = collections.defaultdict(dict)
for w1, w2_counts in bigram_counts.items():
    total_w1_count = sum(w2_counts.values())
    for w2, count in w2_counts.items():
        bigram_probs[w1][w2] = count / total_w1_count
```

Screenshot 5: Bigram Model MLE probability calculation code

```
Bigram Samples:
  P('sorcerers' | 'the') = 0.0033
  P('boy' | 'the') = 0.0072
  P('last' | 'the') = 0.0058
```

Screenshot 6: Calculated Bigram Probability Samples

5. Text Generation Results

Text generation was achieved by implementing weighted random sampling based on the learned probability distributions. Below are the actual outputs generated by both models (target length of 35 words per sample).

5.1. Unigram Generation Samples

```python
def generate_unigram_text(probs, length=30):
    words = list(probs.keys())
    probabilities = list(probs.values())
    # Random selection according to the probabilities
    generated = random.choices(words, weights=probabilities, k=length)
    return " ".join(generated)
```

Screenshot 7: Unigram Text Generation Code

UNIGRAM MODEL OUTPUTS

Example 1: and long envelope frowning the though but <s> said she on dont mcgonagall a </s> </s> a when thirdfloor in touch freckles were back hermione did top reason on im her village bin gringotts eyes

Example 2: things your into the the so everything as youknowwhat as this play is <s> an </s> dursleys known stood bronze taught he their deep told leaves <s> <s> that had and out voldemort <s> it

Example 3: crashed was jordan he <s> least little of <s> had body two harry be he way then afternoon bad malfoy much lived these they treacle up leg him they because would go catch toward <s>

Example 4: there of there </s> pushed his they got thinks <s> well hair shouldnt <s> will <s> of </s> use to said unwrapped buy dropping dont had </s> toward moment went it eyes snape steal to

Example 5: more were the stranger playing </s> won it it at about green sweets to <s> on one subject far a said it you fell so keep all dumbledore <s> so him neville was had that

5.2. Bigram Generation Samples

```python
def generate_bigram_text(probs, start_token="<s>", length=30):
    current_word = start_token
    generated = []

    for _ in range(length):
        # If it is not in the current word model or there is no next word option, start randomly
        if current_word not in probs or not probs[current_word]:
            current_word = random.choice(list(probs.keys()))

        next_words = list(probs[current_word].keys())
        next_probs = list(probs[current_word].values())

        # Select the next word according to its probability
        next_word = random.choices(next_words, weights=next_probs, k=1)[0]
        generated.append(next_word)
        current_word = next_word

    return " ".join(generated)
```

Screenshot 8: Bigram Text Generation Code

BIGRAM MODEL OUTPUTS

Example 1: theyre here somewhere </s> <s> neville scrambling up knowin yer ticket fer hogwarts years </s> <s> hagrid as long fingers under his only things to me id be so much as he was enraged to

Example 2: you up to dumbledore you </s> <s> professor mcgonagall deputy headmistress questions exploded if they all the cold </s> <s> what an hed take someone just got into blackness down </s> <s> keep the back

Example 3: oh sorry george </s> <s> harry not for letter bombs he pelts dumbledore and saw her </s> <s> he flew in the entrance to worry well all </s> <s> that hed killed some fighting is

Example 4: out a unicorn well all all over to leave saying </s> <s> it was no less and he brought with six hoops </s> <s> he was happening down the air dead now piled against the

Example 5: you at last comforting </s> <s> so far had sheared it was still see them </s> <s> and down picked up </s> <s> well so much they sprinted back </s> <s> and the invisibility cloak

6. Discussion

6.1. Architectural Differences in Text Generation (Unigram vs. Bigram)

The fundamental difference between unigram-based and bigram-based text generation lies in the history window and the underlying probability assumptions:

- Unigram Generation: Treats every token independently. The generation process chooses words purely based on their overall global frequency in *"Harry Potter and the Sorcerer's Stone"*. It lacks structural syntax or grammatical memory, operating as a 0-order Markov chain.

- Bigram Generation: Conditions the selection of the current token on the single immediately preceding token, operating as a 1st-order Markov chain. It retains a moving memory window of exactly one word of history.

6.2. Evaluating Realism and Syntactic Fluency

As observed in Section 5 (Text Generation Results), the bigram-generated text is significantly more realistic than the unigram-generated "word salad."

- Unigram Flaws: Unigram strings yield disjointed anomalies such as “the the” so everything or consecutive formatting tags like “ <s> <s> ” that had. Because the model chooses tokens independently, high-frequency words like "the", "and", or <s> are heavily favored to appear adjacent to one another, defying English syntax rules.

- Bigram Success: The bigram model enforces local syntactic dependencies. Because it samples from a distribution conditioned on the previous word, a determiner like "the" will likely be followed by a noun or adjective (e.g., the entrance, the air dead) rather than another determiner or a sentence-boundary token. This closely mimics human sentence structures on a micro-level.

6.3. The Zero Probability Problem and Unseen Events

A major vulnerability of language models trained strictly via Maximum Likelihood Estimation is their reaction to unseen events.

- Unseen Words: If a completely new word (e.g., a modern technical term not present in a fantasy novel) is introduced, the model cannot assign it a position since its count is zero.

- Unseen Word Pairs: A more frequent issue occurs when individual words are known, but they appear in a novel sequence during validation or test tracking—such as the bigram sequence ("hermione", "sprinted"). If this specific pairing never occurred in the book, its joint frequency evaluates to exactly zero.

6.4. Mathematical Basis for MLE Zero Allocations

The reason MLE assigns exactly a zero probability to unseen events stems directly from its objective function. MLE is strictly empirical; it maximizes the likelihood of the training data based *only* on observed frequencies: the numerator becomes zero, causing the entire probability distribution for that sequence to collapse to 0.0000. Mathematically, MLE makes a rigid closed-world assumption: what has not been observed in the training corpus is considered strictly impossible in the language.

6.5. Mitigation Strategies: The Role of Smoothing

To prevent the model from assigning zero probabilities to novel sequences and to resolve the structural dead-ends where text generation halts or loops, Smoothing Technquies must be applied.
Smoothing systematically redistributes a small portion of the probability mass from highly frequent, observed words to completely unseen combinations.

- Add-One (Laplace) Smoothing: This technique adds a uniform pseudo-count of 1 to every possible word pair in the vocabulary.

By shifting the baseline, every single word pair receives a non-zero probability ($P > 0$). This ensures the model remains robust, eliminates zero-division threats during text evaluation, and prevents generation dead-ends when moving between rare tokens.

7. Conclusion

This assignment demonstrates the core implementation and theoretical underpinnings of probabilistic text generation using Maximum Likelihood Estimation (MLE) on a literary corpus (*"Harry Potter and the Sorcerer's Stone"*).

The empirical evaluation of both models yields several critical insights into computational linguistics:

- **Context and Fluency:** Moving from a 0-order Markov chain (Unigram) to a 1st-order Markov chain (Bigram) represents a massive leap in language modeling. While the unigram model produces a chaotic word salad due to its strict independence assumption, the bigram model successfully captures local syntax, resulting in micro-phrases that exhibit human-like grammatical patterns.

- **The Limits of Local Markov Constraints:** Despite the improved realism of the bigram model, it is fundamentally limited by its narrow memory window (one-word history). Neither model is capable of achieving global narrative coherence, semantic intent, or contextual plot consistency over long-term sentences.

- **The MLE Fallacy:** While MLE is computationally straightforward and intuitive to implement via conditional frequency matrices, its strict reliance on empirical frequencies introduces the critical "zero-probability problem." Without architectural modifications, MLE-based models cannot handle unseen events or language variations outside the training scope.

In conclusion, while unigram and bigram models are insufficient on their own for complex or modern creative text generation tasks, they establish the vital fundamental principles of statistical language

processing. To achieve human-level prose, structural depth, and long-term narrative consistency, more advanced NLP methodologies - such as higher-order n-gram models combined with advanced smoothing algorithms (e.g., Kneser-Ney), or deep learning-based Transformer architectures (like GPT networks) - are strictly necessary.